SOFTWARE FAULT PREDICTION USING VARIOUS SUPERVISED MACHINE LEARNING TECHNIQUES

SANGEETA SAHOO

Roll No: 2345013

M.Sc. (Mathematics with Data Science)

Under the Supervision of

Dr. Vishal Pradhan

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

Deemed to be University U/S 3 of the UGC Act, 1956

CONTENTS

- Introduction
- Motivation
- Proposed Methodology
  - Data Preprocessing
  - Correlation and heatmap
  - Classification
- Experimental Analysis
  - Precision, Recall, F1-Score
  - Mean Square Error (MSE)
  - ROC Curve & AUC
- Conclusion
- References

INTRODUCTION

- **Understanding Software Faults**
  - Software faults occur when a system or component behaves unexpectedly.
  - Causes include coding errors, design flaws, or complex interactions.
  - Consequences can range from minor bugs to devastating failures (e.g., aviation, healthcare, finance).

- **Need for Software Fault Prediction**
  - Software is deeply integrated into critical operations across industries.
  - Early fault prediction enhances reliability, safety, and reduces maintenance costs.
  - Predictive approaches help protect end-users and organizational reputation.

INTRODUCTION

- **Real-World Impact of Software Faults**
  - *Aviation*: Flight control glitches affecting passenger safety.
  - *Healthcare*: Medical software faults risking patient lives.
  - *Finance*: Transaction errors, security violations.
  - *Automotive and Public Services*: Operational failures causing risks and delays.

- **Challenges in Fault Prediction**
  - Poor data quality and evolving software environments.
  - Complex system dependencies make fault isolation difficult.
  - Human factors add unpredictability.
  - Continuous updates required for effective predictive models.

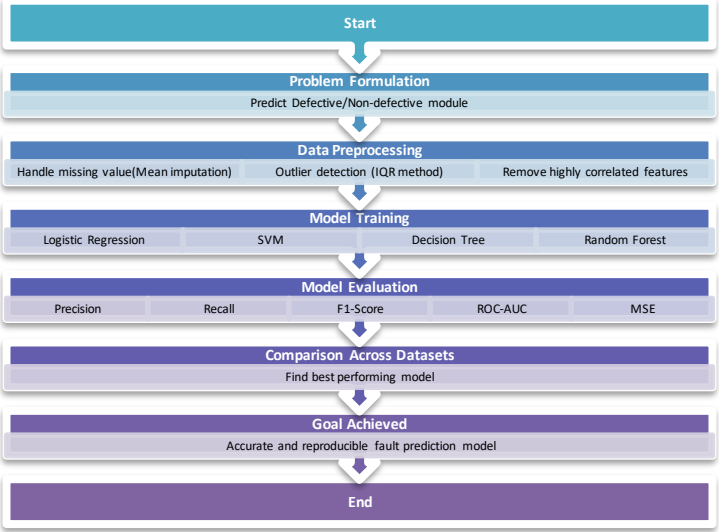
MOTIVATION

With the growing size and complexity of software systems, predicting software faults early has become very important. While many earlier models focused on improving prediction accuracy, they often ignored problems such as:

- Not handling missing values properly.
- Ignoring outliers and feature correlation.
- Testing models only on single datasets, making results less reliable.
- Models that were hard to understand and apply in real-world projects.

This study focuses on solving these problems by using better data cleaning, multiple models, and simple, clear result analysis.

PROPOSED METHODOLOGY



DATA PREPROCESSING

- ***Handle Missing Values***: Filled missing data with column mean.

  (Specifically, in the DATASET 2 dataset, the column DECISION_DENSITY was found to have 139 missing values)

- ***Outlier Detection***: Used-Interquartile Range (IQR) to find and remove extreme values without losing important patterns.

- ***Feature Selection***: Removed highly correlated features (correlation ≥ 0.9) to avoid overlap and improve model learning.

CORRELATION MATRIX & HEATMAP

Correlation Matrix:

A table showing how strongly each pair of features is related.

- Pearson's correlation coefficient (r):
  - ✓ +1: Perfect positive correlation
  - ✓ 0: No correlation
  - ✓ –1: Perfect negative correlation
- Formula:

$$r = \frac{Cov\,(X,Y)}{\sigma_X.\sigma_Y}$$

Heatmap:

A visual tool to display the correlation matrix using colors.

- Color meaning:
  - ✓ Dark red = Strong positive correlation
  - ✓ Dark blue = Strong negative correlation
  - ✓ Light colors = Little or no correlation

GROUP	KEEP	DROP
HALSTEAD_PROG_TIME, HALSTEAD_EFFORT	HALSTEAD_EFFORT	HALSTEAD_PROG_TIME
HALSTEAD_ERROR_EST, HALSTEAD_VOLUME, HALSTEAD_LENGTH, NUM_OPERANDS, NUM_OPERATORS	HALSTEAD_VOLUME	HALSTEAD_ERROR_EST, HALSTEAD_LENGTH, NUM_OPERANDS, NUM_OPERATORS
LOC_EXECUTABLE, LOC_TOTAL	LOC_EXECUTABLE	LOC_TOTAL
NUM_UNIQUE_OPERANDS, NUM_UNIQUE_OPERATORS	NUM_UNIQUE_OPERANDS	NUM_UNIQUE_OPERATORS
DESIGN_COMPLEXITY, CYCLOMATIC_COMPLEXITY (based on typical pairing)	CYCLOMATIC_COMPLEXITY	DESIGN_COMPLEXITY
BRANCH_COUNT, CONDITION_COUNT (assumed common pairing)	CONDITION_COUNT	BRANCH_COUNT

Table 1: *Feature Reduction Based on Correlation Analysis for Dataset 1*

CORRELATION MATRIX & HEATMAP

For Dataset 1:

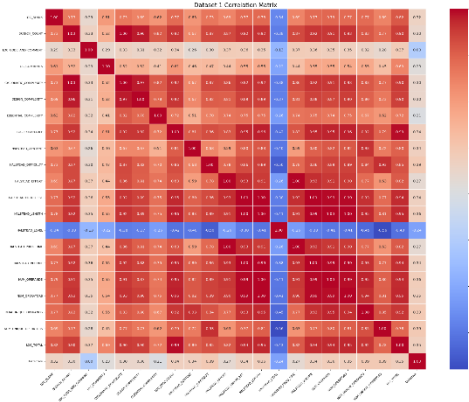


Fig.: *Heat Map of original dataset*

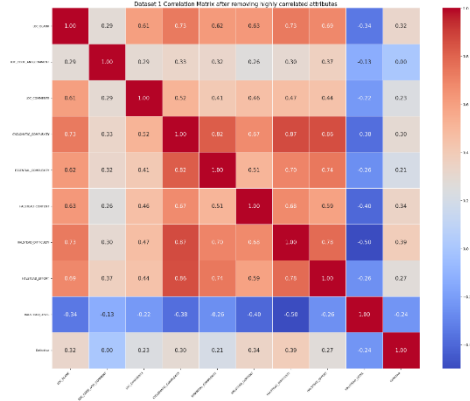


Fig.: *Heat Map of dataset after removing highly correlated attributes*

GROUP	KEEP	DROP
HALSTEAD_EFFORT, HALSTEAD_PROG_TIME, HALSTEAD_LENGTH, HALSTEAD_VOLUME, HALSTEAD_ERROR_EST, NUM_OPERATORS, NUM_OPERANDS, NUM_UNIQUE_OPERANDS	HALSTEAD_EFFORT	HALSTEAD_PROG_TIME, HALSTEAD_LENGTH, HALSTEAD_VOLUME, HALSTEAD_ERROR_EST, NUM_OPERATORS, NUM_OPERANDS, NUM_UNIQUE_OPERANDS
LOC_EXECUTABLE, LOC_TOTAL, NUMBER_OF_LINES, EDGE_COUNT, NODE_COUNT	LOC_EXECUTABLE	LOC_TOTAL, NUMBER_OF_LINES, EDGE_COUNT, NODE_COUNT
CONDITION_COUNT, MODIFIED_CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, DECISION_COUNT, BRANCH_COUNT	CONDITION_COUNT	MODIFIED_CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, DECISION_COUNT, BRANCH_COUNT
CYCLOMATIC_DENSITY, NORMALIZED_CYLOMATIC_COMPLEXITY	CYCLOMATIC_DENSITY	NORMALIZED_CYLOMATIC_COMPLEXITY

Table 2: *Feature Reduction Based on Correlation Analysis for Dataset 2*

CORRELATION MATRIX & HEATMAP

For Dataset 2:

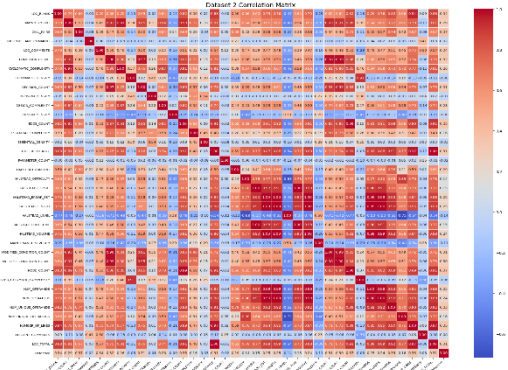


Fig.: *Heat Map of original dataset*

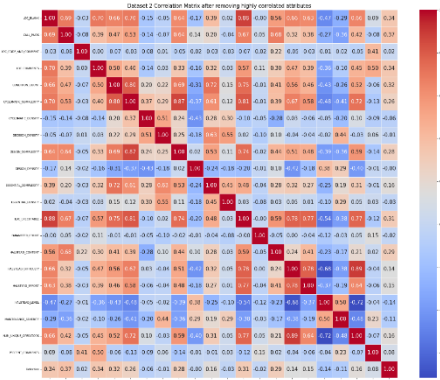


Fig.: *Heat Map of dataset after removing highly correlated attributes*

GROUP	KEEP	DROP
HALSTEAD_EFFORT, HALSTEAD_PROG_TIME	HALSTEAD_EFFORT	HALSTEAD_PROG_TIME
HALSTEAD_VOLUME, HALSTEAD_ERROR_EST, HALSTEAD_LENGTH, NUM_OPERATORS, NUM_OPERANDS	HALSTEAD_VOLUME	HALSTEAD_ERROR_EST, HALSTEAD_LENGTH, NUM_OPERATORS, NUM_OPERANDS
LOC_EXECUTABLE, LOC_TOTAL, NUMBER_OF_LINES	LOC_EXECUTABLE	LOC_TOTAL, NUMBER_OF_LINES
BRANCH_COUNT, CYCLOMATIC_COMPLEXITY, MODIFIED_CONDITION_COUNT, DECISION_COUNT, CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, EDGE_COUNT	CYCLOMATIC_COMPLEXITY	BRANCH_COUNT, MODIFIED_CONDITION_COUNT, DECISION_COUNT, CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, EDGE_COUNT
GLOBAL_DATA_COMPLEXITY (correlated with CYCLOMATIC_COMPLEXITY)	(already included above)	GLOBAL_DATA_COMPLEXITY
HALSTEAD_CONTENT, NUM_UNIQUE_OPERANDS	HALSTEAD_CONTENT	NUM_UNIQUE_OPERANDS

Table 3: *Feature Reduction Based on Correlation Analysis for Dataset 3*

CORRELATION MATRIX & HEATMAP

For Dataset 3:

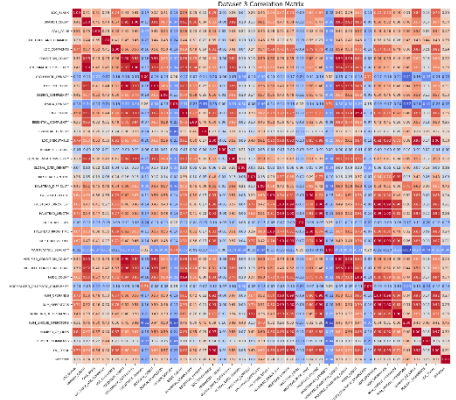


Fig.: *Heat Map of original dataset*

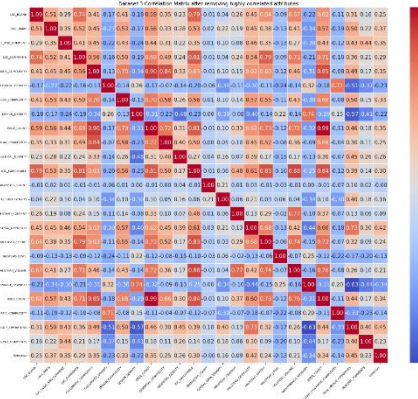


Fig.: *Heat Map of dataset after removing highly correlated attributes*

GROUP	KEEP	DROP
HALSTEAD_EFFORT, HALSTEAD_PROG_TIME	HALSTEAD_EFFORT	HALSTEAD_PROG_TIME
CONDITION_COUNT, MODIFIED_CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, DECISION_COUNT, EDGE_COUNT, NODE_COUNT, BRANCH_COUNT	CONDITION_COUNT	MODIFIED_CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, DECISION_COUNT, EDGE_COUNT, NODE_COUNT, BRANCH_COUNT
HALSTEAD_ERROR_EST, HALSTEAD_VOLUME, HALSTEAD_LENGTH, NUM_OPERATORS, NUM_OPERANDS	HALSTEAD_VOLUME	HALSTEAD_ERROR_EST, HALSTEAD_LENGTH, NUM_OPERATORS, NUM_OPERANDS
LOC_EXECUTABLE, LOC_TOTAL, NUMBER_OF_LINES	LOC_EXECUTABLE	LOC_TOTAL, NUMBER_OF_LINES
CYCLOMATIC_COMPLEXITY, DESIGN_COMPLEXITY	CYCLOMATIC_COMPLEXITY	DESIGN_COMPLEXITY

Table 4: *Feature Reduction Based on Correlation Analysis for Dataset 4*

CORRELATION MATRIX & HEATMAP

For Dataset 4:

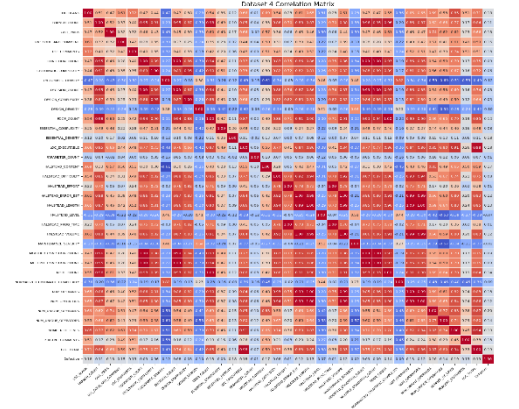


Fig.: *Heat Map of original dataset*

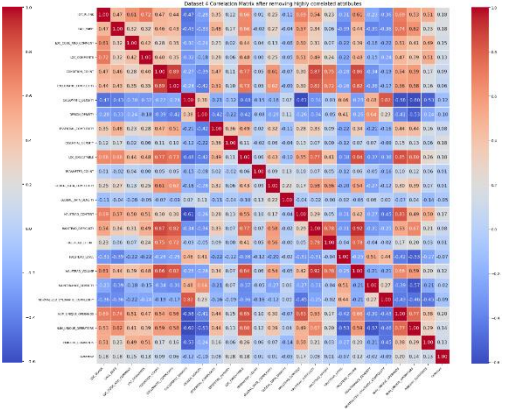


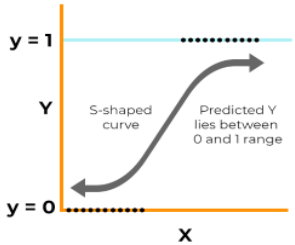
Fig.: *Heat Map of dataset after removing highly correlated attributes*

CLASSIFICATION

Logistic Regression:

- A supervised learning algorithm used for binary classification (Yes/No, Defective/Non-defective).
- Applies the Sigmoid function (Sigmoid = $\frac{1}{1+e^{-z}}$) to squash the output between 0 and 1.

  (Where z = $w_0 + x_1 w_1 + x_2 w_2 + \ldots + x_n w_n$)
- If probability > 0.5 → Class 1
- If probability ≤ 0.5 → Class 0

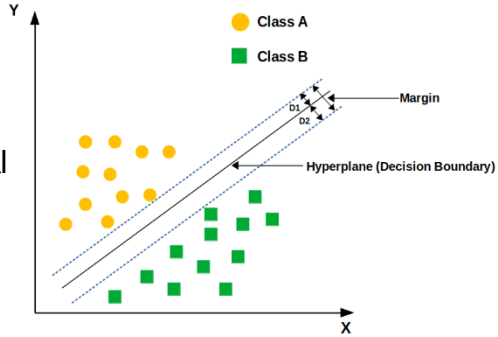


CLASSIFICATION

Support Vector Machine(SVM):

A supervised learning algorithm used for classification tasks.

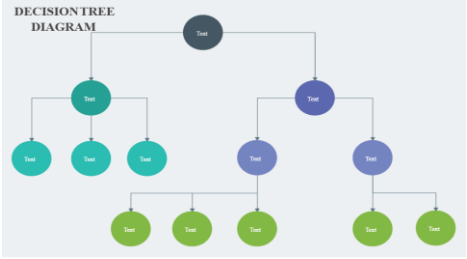
- Finds the best boundary (hyperplane) to separate classes.
- Good for high-dimensional data
- Stable against overfitting



CLASSIFICATION

Decision Tree :

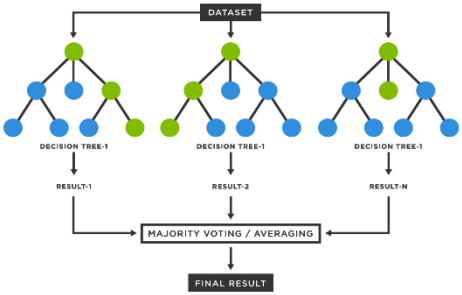
- A supervised learning algorithm that models decisions in a tree-like structure, breaking data into smaller subsets based on attributes.
- Easy to understand, interpret, and visualize. Works well for classification and regression tasks.
- Consists of nodes — root, internal nodes (decision nodes), and leaf nodes (outcomes).



CLASSIFICATION

Random Forest :

- Ensemble method combining multiple decision trees for stronger predictions.
- Each tree votes, and the class with the majority votes is chosen.
- Reduces overfitting by averaging results from multiple trees.
- Handles large datasets with many features, maintaining high accuracy.
- Strong and reliable for classification tasks in diverse scenarios.
- Works well even with missing data or noisy features.



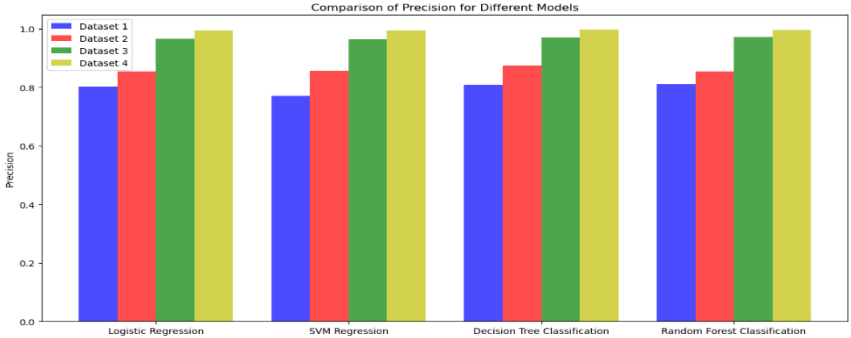
EXPERIMENTAL ANALYSIS

	Actual: Defective (1)	Actual: Not Defective (0)
Predicted: Defective (1)	TP (True Positive) – Correctly predicted defect	FP (False Positive) – Incorrectly flagged as defective
Predicted: Not Defective (0)	FN (False Negative) – Missed actual defect	TN (True Negative) – Correctly predicted as non-defective

EXPERIMENTAL ANALYSIS

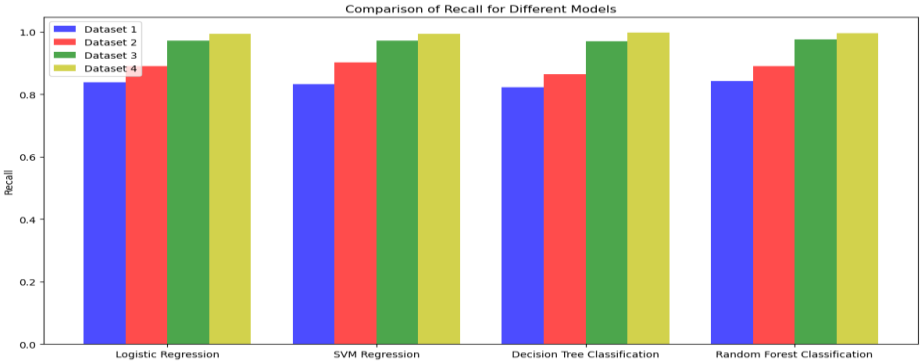
Precision measures the accuracy of positive predictions. It is calculated as:

$$\text{Precision} = \frac{TP}{TP+FP}$$



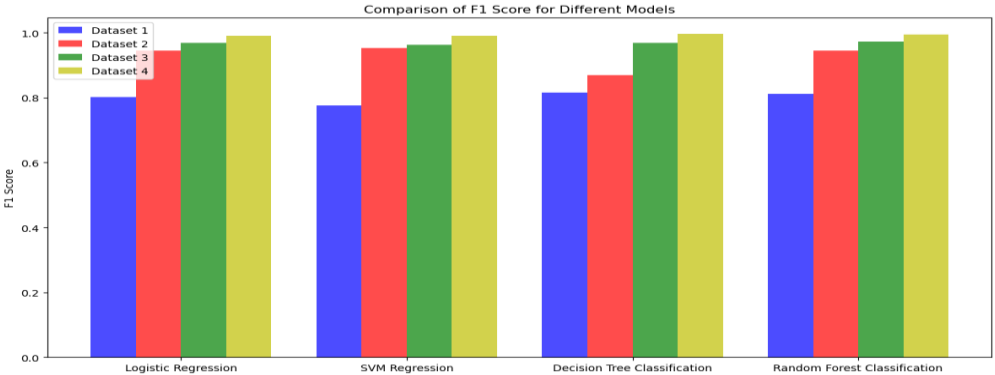
Recall measures the ability to identify all the positive instances (defective files). It is calculated as:

$$\text{Recall} = \frac{TP}{TP+FN}$$



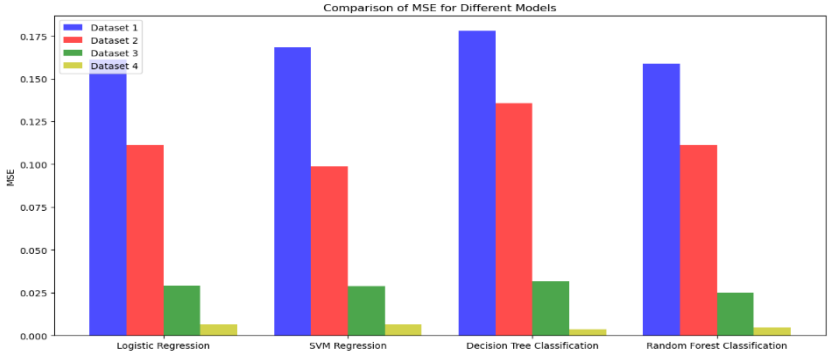
F-measure (F1-score) is a balanced harmonic mean of Precision and Recall, providing a single metric to evaluate the model's performance. It is calculated as:

$$\text{F-measure} = \frac{2 * Precision * Recall}{Precision + Recall}$$



Mean Square Error (MSE) measures the average squared difference between actual and predicted values. A lower MSE means better performance, and larger errors are penalized more.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$



ROC CURVE AND AUC

ROC (Receiver Operating Characteristic) Curve

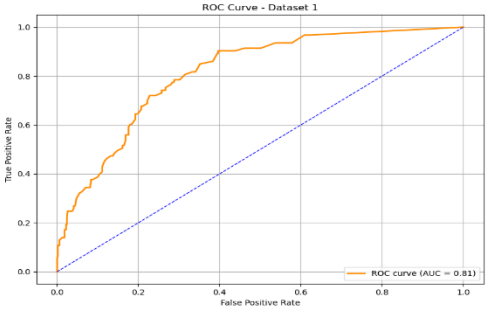
- ROC (Receiver Operating Characteristic) curve plots the True Positive Rate (TPR) vs. the False Positive Rate (FPR).
- The curve helps evaluate classifier performance at different thresholds.

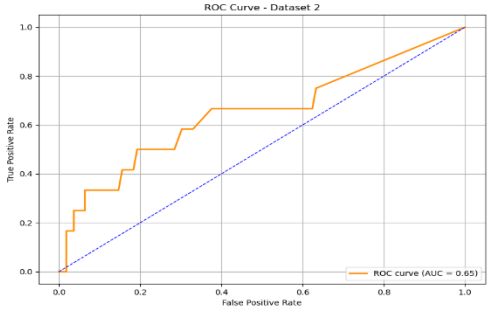
AUC (Area Under the Curve)

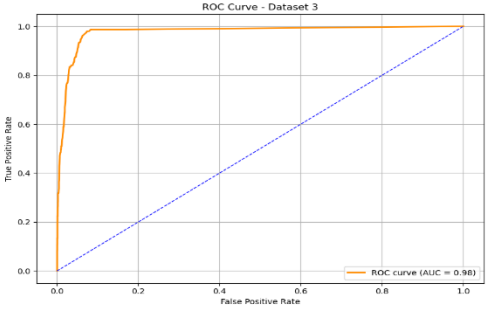
AUC measures the overall performance of a classifier which represents the area under the ROC curve.

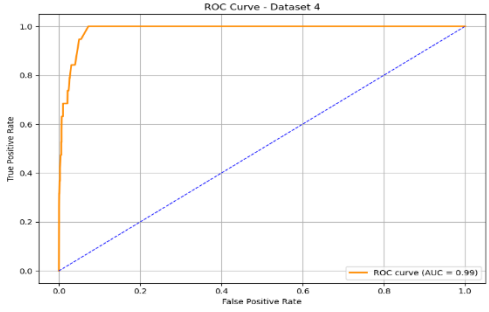
- AUC = 1: Perfect prediction
- AUC = 0.5: No discrimination (random)
- AUC < 0.5: Worse than random

Higher AUC values indicate a better model for distinguishing between positive and negative classes.


ROC Curve - Dataset 1
True Positive Rate
False Positive Rate
ROC curve (AUC = 0.81)


ROC Curve - Dataset 2
True Positive Rate
False Positive Rate
ROC curve (AUC = 0.65)


ROC Curve - Dataset 3
True Positive Rate
False Positive Rate
ROC curve (AUC = 0.98)


ROC Curve - Dataset 4
True Positive Rate
False Positive Rate
ROC curve (AUC = 0.99)

CONCLUSION

- Supervised machine learning techniques can effectively predict software faults using static code metrics.
- An effective data preparation process including outlier handling, feature selection, and missing value imputation significantly boosts model performance.
- Among the evaluated models, Random Forest consistently outperformed others across multiple datasets.
- Proper data handling and algorithm selection are crucial for achieving high precision, recall, and low error rates in software defect prediction.
- This study confirms that machine learning can play a vital role in improving software reliability, minimizing risks, and reducing maintenance costs.

REFERENCES

- Tang, Y., Dai, Q., Yang, M., Du, T., & Chen, L. (2023). Software defect prediction ensemble learning algorithm based on adaptive variable search algorithm. *International Journal of Machine Learning and Cybernetics, 14*(6), 1967-1987.
- Aquil, Ishak, Predicting software defects using machine learning techniques, IJATCSE 9 (4) (2020) 6609–6616.
- Ghotra, B., McIntosh, S., & Hassan, A. E. (2015) Revisiting the impact of classification techniques on defect prediction models. IEEE/ACM ICSE.
- Pedregosa, F., et al. (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research.
- Radjenović, D., Heričko, M., Torkar, R., & Živkovič, A. (2013) Software fault prediction metrics: A systematic literature review. Information and Software Technology.
- NASA Metrics Data Program (MDP) datasets (2010+) Available at: https://mdp.ivv.nasa.gov/
- Geeks for geeks https://www.geeksforgeeks.org/


thank you